1) The range of PC counts "n" is not justified. 9/10
2) OK 10/10
3) The range of cluster counts "m" is not justified. 8/10
4) Excellent 40/40
5) OK 15/15
6) An error in making error matrices 4/5
7) OK 10/10

96

Schultz_S

March 17, 2026

[1]:
```
#First Import the Train and Test Data
import pandas as pd
import numpy as np

train_data = pd.read_csv('training.csv')
testing_data = pd.read_csv('testing.csv')
```

[2]:
```
train_data.head()
```

[2]:
```
       f1      f2      f3      f4      f5      f6      f7      f8      f9  \
0 -0.4394 -0.0930  0.1718  0.4620  0.6226  0.4704  0.3578  0.0478 -0.1184
1 -0.4348 -0.1198  0.2474  0.4036  0.5026  0.6328  0.4948  0.0338 -0.0520
2 -0.2330  0.2124  0.5014  0.5222 -0.3422 -0.5840 -0.7168 -0.6342 -0.8614
3 -0.3808 -0.0096  0.2602  0.2554 -0.4290 -0.6746 -0.6868 -0.6650 -0.8410
4 -0.3412  0.0946  0.6082  0.6216 -0.1622 -0.3784 -0.4324 -0.4358 -0.4966

      f10  …    f609    f610    f611    f612    f613    f614    f615  \
0 -0.2310  …  0.4102  0.2052  0.3846  0.3590  0.5898  0.3334  0.6410
1 -0.1302  …  0.0000  0.2954  0.2046  0.4772  0.0454  0.2046  0.4318
2 -0.8318  … -0.1112 -0.0476 -0.1746  0.0318 -0.0476  0.1112  0.2540
3 -0.9614  … -0.0504 -0.0360 -0.1224  0.1366  0.2950  0.0792 -0.0072
4 -0.5406  …  0.1562  0.3124  0.2500 -0.0938  0.1562  0.3124  0.3124

     f616    f617  Letter
0  0.5898 -0.4872       1
1  0.4546 -0.0910       1
2  0.1588 -0.4762       2
3  0.0936 -0.1510       2
4  0.2188 -0.2500       3

[5 rows x 618 columns]
```

[3]:
```
#separate data into train X & y
y = np.ravel(train_data.loc[:, ["Letter"]])
X = train_data.loc[:, train_data.columns != 'Letter']
```

[4]:
```
unique_vals, counts = np.unique(y, return_counts=True)
counts #we have a perfectly balanced dataset
```

[4]:
```
array([240, 240, 240, 240, 240, 238, 240, 240, 240, 240, 240, 240, 240,
       240, 240, 240, 240, 240, 240, 240, 240, 240, 240, 240, 240, 240],
      dtype=int64)
```

[5]:
```
#Now its time to build our classifiers:
#1)
#One model must combine support vector machine (SVM) and principal component␣
↪decomposition.
#Use the first "n" principal components of the input features, with n<617, as␣
↪input to the SVM
#and provide quantitative criteria to inform the selection of n.
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.decomposition import PCA

svc_pca = Pipeline([
    ("pca", PCA()),
    ("svc", SVC())
])
```

[6]:
```
#2)
#One model must combine Logistic Regression with stochastic gradient descent (i.
↪e., use the class "SGDClassifier()"
#in Scikit-Learn with the appropriate value for the loss function "loss")
# and principal component decomposition. Use the first "n" principal components␣
↪of the input features,
# with n<617, as input to the SGDClassifier and provide quantitative criteria␣
↪to inform the selection of n.
from sklearn.linear_model import SGDClassifier

sgd_pca = Pipeline([
    ("pca", PCA()),
    ("sgd", SGDClassifier(loss='log_loss', n_jobs=-1))
])
```

[7]:
```
#3)
#One model must combine support vector machine (SVM) and K-Means clustering.
# Specifically, apply K-Means to partition the dataset in "m" clusters and make␣
↪the SVM operate on the
# cluster distances (i.e., distance of each data point from these m clusters).
# Provide a quantitative criterion for your selection of m, e.g.,
# you could maximize a metric of your choosing that assesses either the␣
↪goodness of the partition
```

```python
# or the goodness of the resultant classification.
# TIP: See Chapter 9 in your textbook for relevant examples
from sklearn.cluster import KMeans

svc_kmean = Pipeline([
    ("kmeans", KMeans()),
    ("svc", SVC())
])
```

[8]:
```python
#4)
#Use data in training.csv to optimize the number of principal components,
# the number of clusters, and at least two (2) meaningful hyperparameters␣
↪defining the SVM and SGD classifiers,
# with a minimum of 30 different combinations of hyperparameter values tested␣
↪per model.
# TIP: You can combine PCA (or K-Means) and a classifier in just one object by␣
↪using the utility "Pipeline()"
# in Scikit-Learn. See Chapter 8 in your textbook for relevant examples

from sklearn.model_selection import GridSearchCV

gs_svc_pca = GridSearchCV(
    estimator=svc_pca,
    param_grid= {
        'pca__n_components': [10, 25, 50, 100, 200],
        'svc__C': [0.01, 0.1, 1, 10, 100],
        'svc__kernel': ['linear', 'poly', 'rbf'],
    },
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
    refit=True #I Checked the Documentation
    #This Automatically Refits the best estimator on the entire training set␣
↪after training
    #Which completes criteria 5)
)

gs_sgd_pca = GridSearchCV(
    estimator=sgd_pca,
    param_grid={
        'pca__n_components':[10, 25, 50, 100, 200],
        'sgd__alpha': [1e-3, 1e-2, 1e-1, 1, 10],
        'sgd__penalty': ['l1', 'l2', 'elasticnet']
    },
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
```

Very low values... why would you consider them?

75 combos

odd choice... why?

you got it!

75 combos

```
    refit=True
)

gs_svc_pca.fit(X, y)
print('Grid Search Complete!')

gs_sgd_pca.fit(X, y)
print('Grid Search Complete!')
```

```
Grid Search Complete!
Grid Search Complete!
```

[ ]:
```
gs_svc_kmean = GridSearchCV(
    estimator=svc_kmean,
    param_grid={
        'kmeans__n_clusters': [2, 4, 6, 8, 10, 15, 20],
        'svc__C': [0.01, 0.1, 1, 10, 100],
        'svc__kernel': ['linear', 'poly', 'rbf'],
    },
    cv=5,
    scoring='f1_macro',
    error_score='raise',
    n_jobs=-1,
    refit=True
)
```

How do you know that these clusters are enough?

105 combos

[10]:
```
gs_svc_kmean.fit(X, y)
```

[10]:
```
GridSearchCV(cv=5, error_score='raise',
             estimator=Pipeline(steps=[('kmeans', KMeans()), ('svc', SVC())]),
             n_jobs=-1,
             param_grid={'kmeans__n_clusters': [2, 4, 6, 8, 10, 15, 20],
                         'svc__C': [0.01, 0.1, 1, 10, 100],
                         'svc__kernel': ['linear', 'poly', 'rbf']},
             scoring='f1_macro')
```

[15]:
```
#5)
#  Upon completion of the optimization procedures,
# train the optimal models on the entire training set (i.e., data from training.
 ↪csv)
#  and test their performance on data from testing.csv (test data).

best_svc_pca = gs_svc_pca.best_estimator_
best_sgd_pca = gs_sgd_pca.best_estimator_
best_svc_kmean = gs_svc_kmean.best_estimator_

best_svc_pca
```

[15]: Pipeline(steps=[('pca', PCA(n_components=200)), ('svc', SVC(C=10))])

[16]:
```
best_sgd_pca
```

[16]:
```
Pipeline(steps=[('pca', PCA(n_components=200)),
                ('sgd',
                 SGDClassifier(alpha=0.001, loss='log_loss', n_jobs=-1))])
```

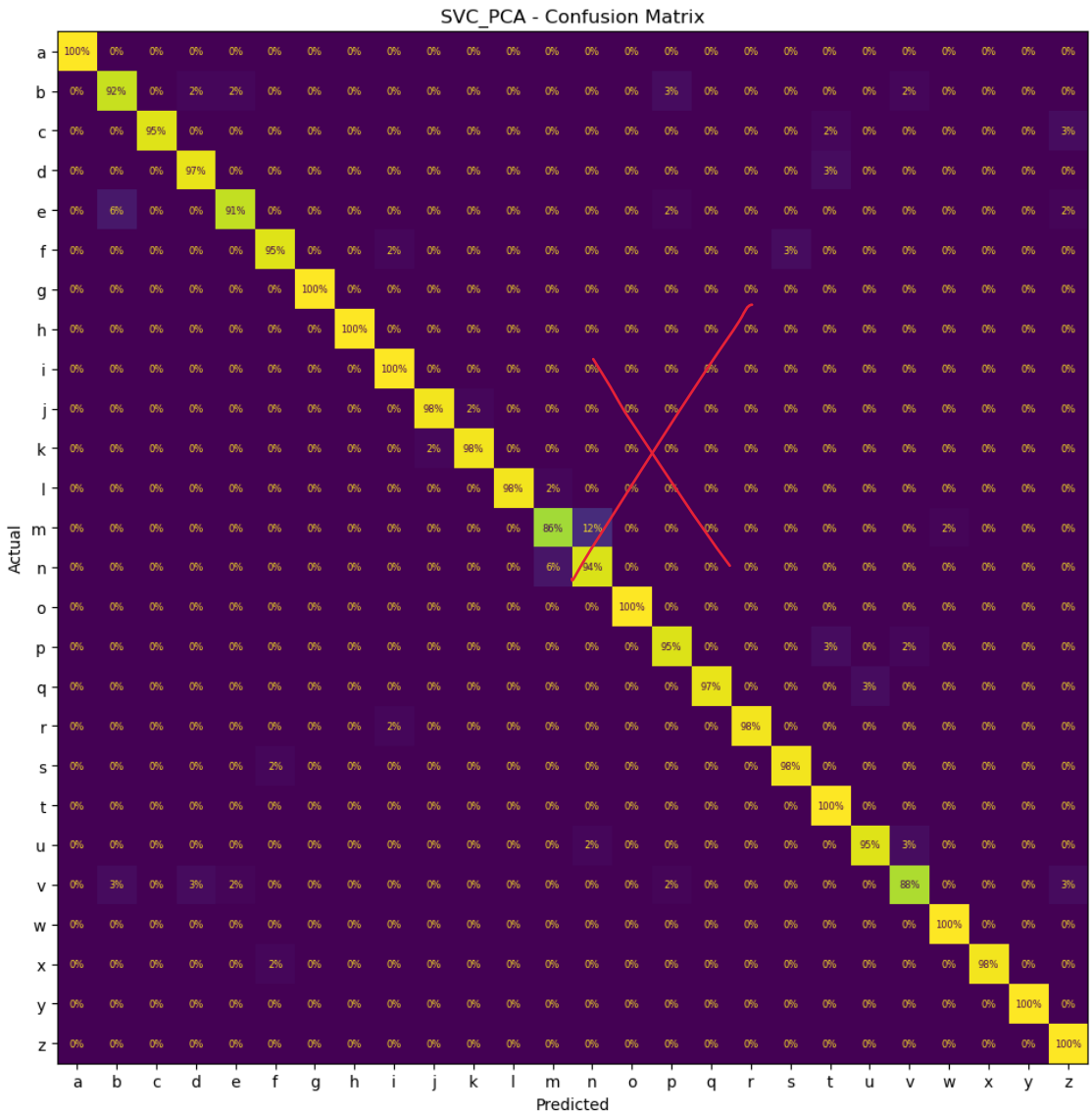
[17]:
```
best_svc_kmean
```

[17]:
```
Pipeline(steps=[('kmeans', KMeans(n_clusters=20)),
                ('svc', SVC(C=1, kernel='poly'))])
```

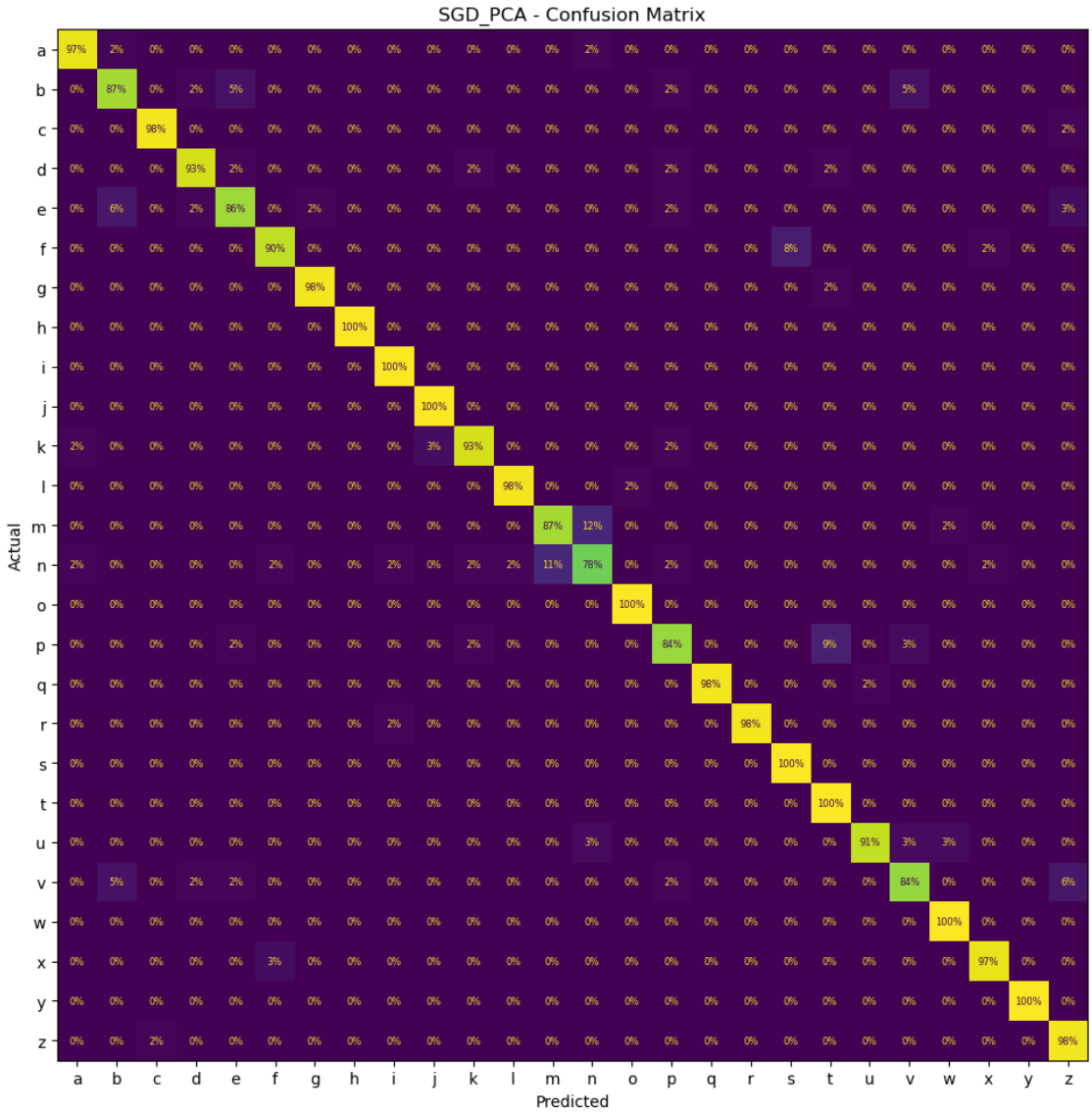
Each Grid Search Resulted in the highest number for the dimensionality reduction. This is not too suprising as we can expect to lose information as dimensions decrease. It's possible scaling features up even more may lead to better results

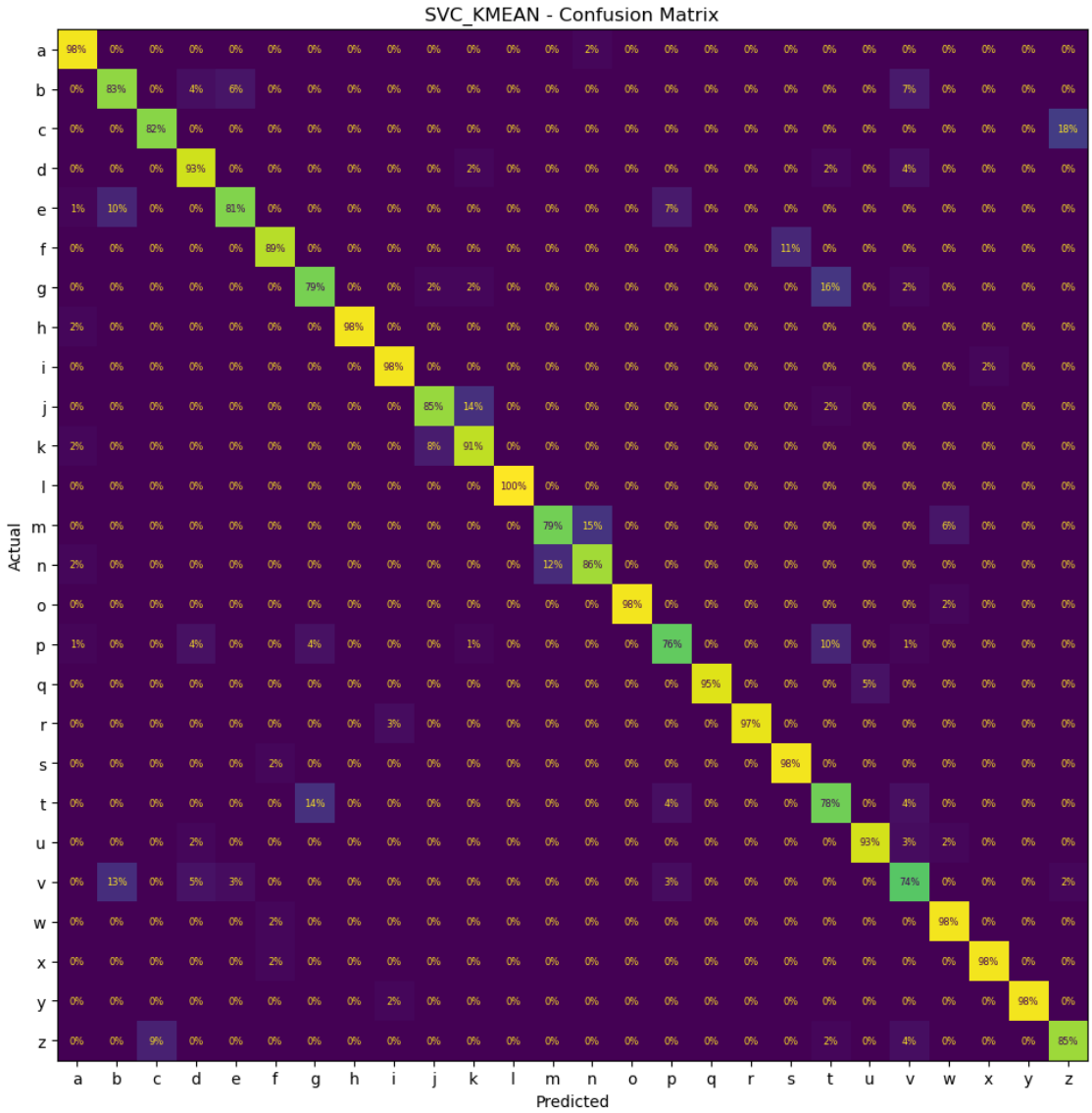
[12]:
```
# 6)
#  Use the predictions on test data to calculate and analyze the error matrix␣
 ↪for all models.
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
import string

y_test = np.ravel(testing_data.loc[:, ["Letter"]])
X_test = testing_data.loc[:, train_data.columns != 'Letter']


for clf in [best_svc_pca, best_sgd_pca, best_svc_kmean]:

    if clf == best_svc_pca:
        name = 'SVC_PCA'
    elif clf == best_sgd_pca:
        name = 'SGD_PCA'
    else:
        name = 'SVC_KMEAN'

    preds = clf.predict(X_test)
    cm = confusion_matrix(preds, y_test, normalize='true')

    fig, ax = plt.subplots(figsize=(10, 10))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,␣
 ↪display_labels=list(string.ascii_lowercase))
    disp.plot(ax=ax, values_format='.0%', colorbar=False, text_kw={"fontsize":␣
 ↪6})
```

```
    ax.set_title(f"{name} - Confusion Matrix")
    ax.set_xlabel(f"Predicted")
    ax.set_ylabel(f'Actual')

    plt.tight_layout()
    plt.show()
```



Error matrices do NOT have the main diagonal...


SGD_PCA - Confusion Matrix
Actual
Predicted
a b c d e f g h i j k l m n o p q r s t u v w x y z



[ ]:

```
```

Overall all the clfs have performed quite well, SVC_PCA stands out especially for its incredible prediction accuracy SVC_KMean while still strong seems to be lagging behind the other two. Unsuprisingly M & n seem to get mixed up quite a lot. However Kmeans is also mistaking Z & C quite a lot

[13]:

```
#    Compare the performance of all models on test data in terms of
# accuracy, precision, recall, ROC curve, and AUC.
# Then, identify the most promising design among the proposed models.

#Let us start with Accuracy, percision, & recall
```

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score

results = dict()

for clf in [best_svc_pca, best_sgd_pca, best_svc_kmean]:

    preds = clf.predict(X_test)

    if clf == best_svc_pca:
        name = 'SVC_PCA'
    elif clf == best_sgd_pca:
        name = 'SGD_PCA'
    else:
        name = 'SVC_KMEAN'

    results[name] = dict()

    acc = accuracy_score(preds, y_test)
    prec = precision_score(preds, y_test, average='macro')
    #We are macro averaging since the train set is perfectly balanced
    recall = recall_score(preds, y_test, average='macro')

    results[name]['acc'] = f'{acc:.2f}'
    results[name]['prec'] = f'{prec:.2f}'
    results[name]['recall'] = f'{recall:.2f}'

results
```

[13]:
```
{'SVC_PCA': {'acc': '0.97', 'prec': '0.97', 'recall': '0.97'},
 'SGD_PCA': {'acc': '0.94', 'prec': '0.94', 'recall': '0.95'},
 'SVC_KMEAN': {'acc': '0.90', 'prec': '0.90', 'recall': '0.90'}}
```

Once again SVC_PCA comes out on top with SGD_PCA trailing shortly behind and SVC_Kmean last Since this dataset is perfectly balanced acc = macro prec = macro recall

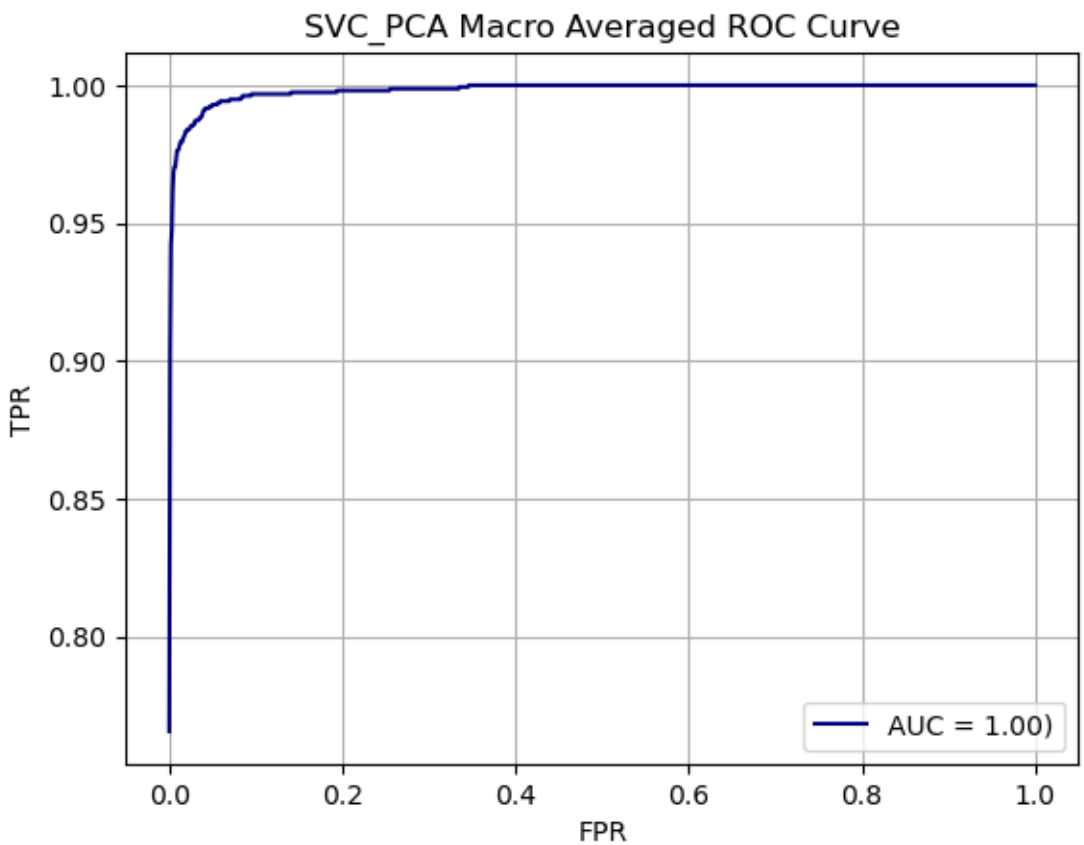
[14]:
```python
#Now Roc Curves
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import LabelBinarizer

label_binarizer = LabelBinarizer()
label_binarizer.fit(y)
y_binned_test = label_binarizer.transform(y_test)

n_classes = y_binned_test.shape[1]

print(y_binned_test.shape)
```

label_binarize (function) is preferred to LabelBinarizer (class) because it allows us to control for the order of the class values

```
for clf in [best_svc_pca, best_sgd_pca, best_svc_kmean]:
    scores = clf.decision_function(X_test)

    print(scores.shape)

    if clf == best_svc_pca:
        name = 'SVC_PCA'
    elif clf == best_sgd_pca:
        name = 'SGD_PCA'
    else:
        name = 'SVC_KMEAN'

    results[name]['tpr'], results[name]['fpr'], results[name]['roc_auc'] =␣
↪dict(), dict(), dict()

    for i in range(n_classes):
        results[name]['fpr'][i], results[name]['tpr'][i], _ =␣
↪roc_curve(y_binned_test[:, i], scores[:, i])
        results[name]['roc_auc'][i] = auc(results[name]['fpr'][i],␣
↪results[name]['tpr'][i])

    fpr_grid = np.linspace(0.0, 1.0, 1000)

    mean_tpr = np.zeros_like(fpr_grid)

    for i in range(n_classes):
        mean_tpr += np.interp(fpr_grid, results[name]['fpr'][i],␣
↪results[name]['tpr'][i])

    mean_tpr /= n_classes

    results[name]["fpr"]["macro"] = fpr_grid
    results[name]["tpr"]["macro"] = mean_tpr
    results[name]['roc_auc']['macro'] = auc(results[name]["fpr"]["macro"],␣
↪results[name]["tpr"]["macro"])

    plt.plot(
        results[name]["fpr"]["macro"],
        results[name]["tpr"]["macro"],
        label=f"AUC = {results[name]['roc_auc']['macro']:.2f})",
        color="navy"
    )

    plt.xlabel("FPR")
    plt.ylabel("TPR")
    plt.title(f'{name} Macro Averaged ROC Curve')
    plt.legend(loc='lower right')
```

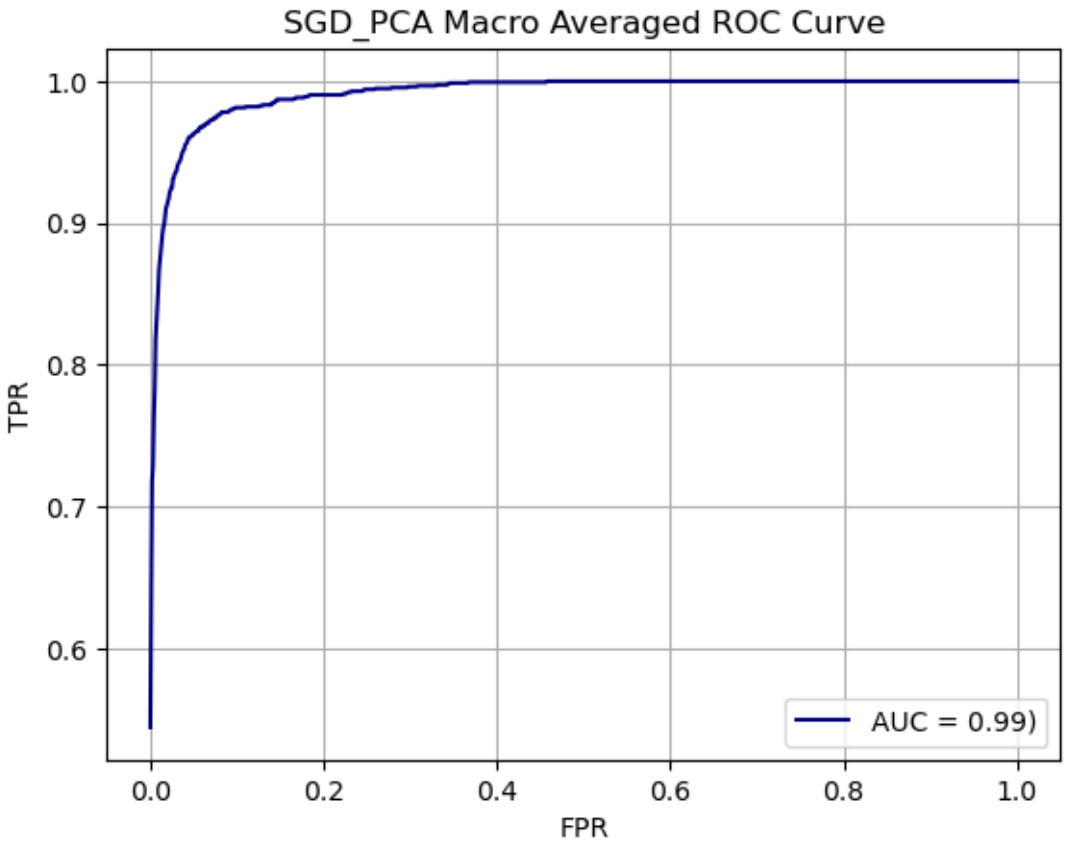
```
    plt.grid(True)

    plt.show()
```

```
(1559, 26)
(1559, 26)
```

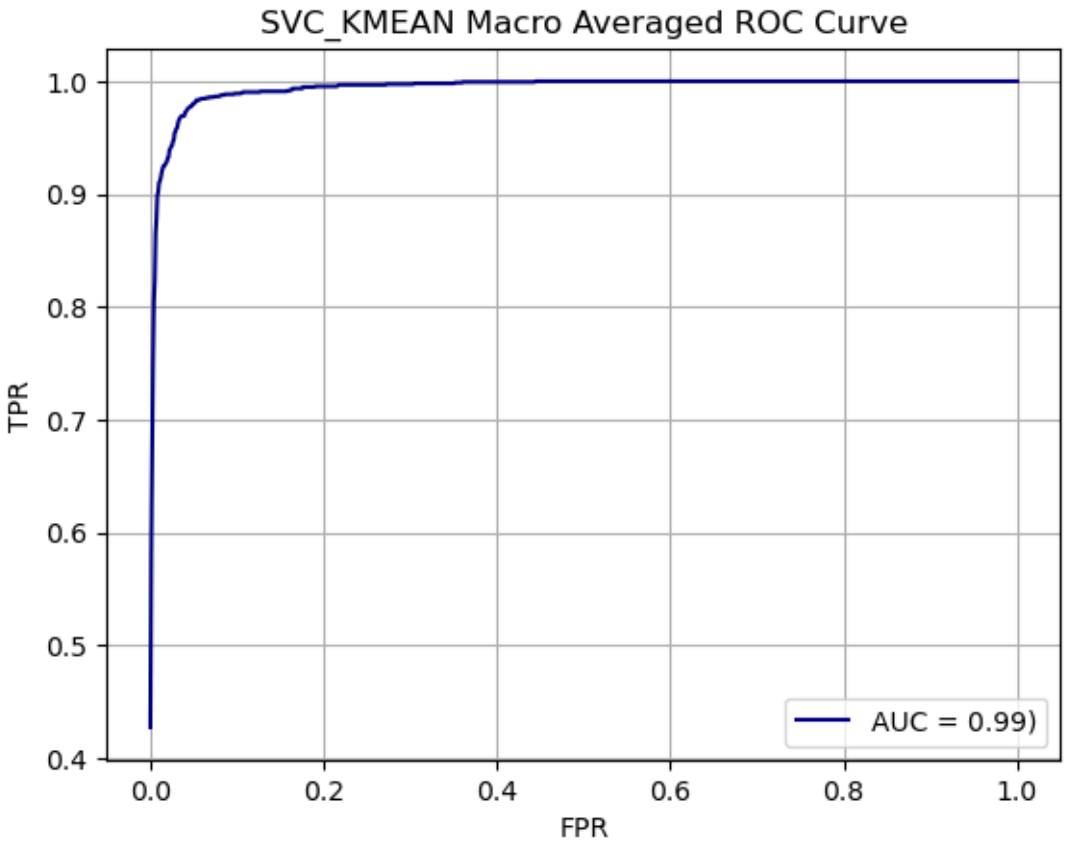


Make all ROC curves in one plot to facilitate the comparison

```
(1559, 26)
```


SGD_PCA Macro Averaged ROC Curve
1.0
0.9
0.8
0.7
0.6
TPR
0.0
0.2
0.4
0.6
0.8
1.0
FPR
AUC = 0.99)

```
(1559, 26)
```



SVC_PCA Performs slightly better in this regard, though all do quite well. Due to a superior performance in both the Confusion Matrix, Accuracy, and AUC. I would select the SVC_PCA Pipeline. From here, I would continue to find tune the hyper parameters of SVC_PCA to try to get the best performance achievable.